



The Only Notation You Actually Need

A hands-on workbook for PyTorch & NumPy practitioners — from dot products to multi-head attention in one unified notation.

Complete every task. Run every code block. Don't move on until it executes.

1 Why einsum?

Most tensor operations — dot products, matrix multiplications, attention scores, covariances, tensor contractions — are **index contraction problems**. NumPy and PyTorch each have a dozen specialized functions for these (matmul, tensordot, bmm, trace, outer...), but **one einsum string encodes all of them identically**. Once you think in indices, you never reach for np.dot vs @ vs bmm again.

■ einsum = Einstein summation convention, invented in 1916 to make GR equations compact. It says: repeated indices are summed over, free indices survive to the output.

1.1 The grammar in one rule

An einsum string looks like 'ij,jk->ik'. Read it as: **for every combination of free indices (i, k), sum over all contracted indices (j)**. That's it.

```
# The three zones of an einsum string
#
# input subscripts output subscripts
# ██████████ ██████████
# 'ij , jk -> ik'
# ↑↑↑
# A's B's result
# axes axes
#
# Repeated index 'j' does NOT appear in output → it is summed away.
# Free indices 'i','k' appear in output → they are kept.
```

1.2 The two rules that cover everything

Rule 1 (contraction): Any index that appears on the left but not on the right is summed over.

Rule 2 (free indices): Any index that appears on the right defines a dimension of the output tensor.

■ Order of output indices matters! 'ij,jk->ik' gives (m,p) but 'ij,jk->ki' transposes the result to (p,m).

2 Anatomy: Eight Core Patterns

Pattern 1 — Dot product (scalar output)

Both vectors share the same index i ; nothing survives to output $\rightarrow$ scalar.

```
import numpy as np, torch

a = np.array([1., 2., 3.])
b = np.array([4., 5., 6.])

# Conventional
print(np.dot(a, b)) # 32.0

# einsum
print(np.einsum('i,i->', a, b)) # 32.0
# Read: sum over i of a[i]*b[i]
```

Pattern 2 — Outer product (matrix from two vectors)

Indices i and j are independent; no contraction $\rightarrow$ full outer product.

```
# a has index 'i', b has index 'j'
# They share NO index  $\rightarrow$  no summation
# Output 'ij'  $\rightarrow$  a 2D matrix

result = np.einsum('i,j->ij', a, b)
# result[i,j] = a[i] * b[j]
print(result.shape) # (3, 3)
```

Pattern 3 — Matrix-vector multiply

```
A = np.random.randn(4, 3)
v = np.random.randn(3)

# 'ij,j->i' : sum over j (columns of A)
y = np.einsum('ij,j->i', A, v)
assert np.allclose(y, A @ v)
print(y.shape) # (4,)
```

Pattern 4 — Matrix-matrix multiply

```
A = np.random.randn(4, 3)
B = np.random.randn(3, 5)

# 'ij,jk->ik' : sum over j (inner dim)
C = np.einsum('ij,jk->ik', A, B)
assert np.allclose(C, A @ B)
print(C.shape) # (4, 5)
```

Pattern 5 — Trace (diagonal sum)

```
M = np.random.randn(4, 4)
```

```
# 'ii->' : same index for both dims → diagonal elements, then sum
t = np.einsum('ii->', M)
assert np.isclose(t, np.trace(M))
```

Pattern 6 — Row sum and column sum

```
A = np.random.randn(3, 4)

row_sums = np.einsum('ij->i', A) # sum over j (columns)
col_sums = np.einsum('ij->j', A) # sum over i (rows)

assert np.allclose(row_sums, A.sum(axis=1))
assert np.allclose(col_sums, A.sum(axis=0))
```

Pattern 7 — Batch matrix multiply (3D)

The **b** index is free — it just rides through. Contraction happens over **k**.

```
import torch

A = torch.randn(8, 4, 3) # batch=8, m=4, k=3
B = torch.randn(8, 3, 5) # batch=8, k=3, n=5

C = torch.einsum('bik,bkn->bin', A, B)
assert torch.allclose(C, torch.bmm(A, B))
print(C.shape) # torch.Size([8, 4, 5])
```

Pattern 8 — Scaled dot-product attention (QKV)

The workhorse of every transformer. Four indices: **batch**, **head**, sequence position (**i**, **j**), head **dim**.

```
B, H, N, D = 2, 4, 16, 32
Q = torch.randn(B, H, N, D)
K = torch.randn(B, H, N, D)

# Attention logits: for each (b,h), compute N×N matrix
# Contract over d (head dim)
scores = torch.einsum('bhid,bhjd->bhij', Q, K) / D**0.5
print(scores.shape) # torch.Size([2, 4, 16, 16])
```

■ This is exactly the scaled dot-product inside `nn.MultiheadAttention` before the softmax.

3 Task Sheet (implement before reading on)

For each task: (1) write the einsum string, (2) run it, (3) verify with the conventional call, (4) tick off.

Task 1 — Inner product of two 5-D vectors

`a = torch.randn(5)` `b = torch.randn(5)` — Compute the scalar $a \cdot b$.

Write your einsum string: _____ Expected answer: `torch.dot(a,b)` → scalar

Task 2 — Transpose a 2D matrix

`A = torch.randn(3, 7)` — Produce shape (7, 3) using einsum.

Write your einsum string: _____ Expected answer: `'ij->ji'` → (7,3)

Task 3 — Element-wise multiply, then sum

`A = torch.randn(4,4)` `B = torch.randn(4,4)` — Compute the Frobenius inner product: sum of all element-wise products (a scalar).

Write your einsum string: _____ Expected answer: `'ij,ij->'` → scalar

Task 4 — Outer product: 2 vectors → matrix

`u = torch.randn(6)` `v = torch.randn(8)` — Produce a (6,8) matrix where $\text{result}[i,j] = u[i] * v[j]$.

Write your einsum string: _____ Expected answer: `'i,j->ij'` → (6,8)

Task 5 — Diagonal of a square matrix

`A = torch.randn(5,5)` — Extract the main diagonal as a vector (shape (5,)) without using `torch.diag`.

Write your einsum string: _____ Expected answer: `'ii->i'` → (5,)

Task 6 — Matrix trace via einsum

`A = torch.randn(6,6)` — Compute the scalar trace (sum of diagonal) using einsum.

Write your einsum string: _____ Expected answer: `'ii->'` → scalar

Task 7 — Row-wise L2 norm squared

`X = torch.randn(10, 4)` — For each row i , compute $\sum_j X[i,j]^2$. Output shape: (10,). Hint: a matrix times itself.

Write your einsum string: _____ Expected answer: `'ij,ij->i'` → (10,)

Task 8 — Batch dot products

`A = torch.randn(32, 64)` `B = torch.randn(32, 64)` — For each of the 32 samples, compute the dot product of the two 64-D vectors. Output: (32,).

Write your einsum string: _____ Expected answer: `'bi,bi->b'` → (32,)

Task 9 — 3-tensor contraction

`A = torch.randn(3, 4, 5)` `B = torch.randn(4, 5, 6)` — Contract (sum) over both j and k . Output shape: (3, 6).

Write your einsum string: _____ Expected answer: `'ijk,jkl->il'` → (3,6)

Task 10 — Bilinear form $x^T W y$

$x = \text{torch.randn}(4)$ $W = \text{torch.randn}(4,4)$ $y = \text{torch.randn}(4)$ — Compute the scalar $x^T W y$ in ONE einsum call (no intermediate variables).

Write your einsum string: _____ Expected answer: 'i,ij,j->' $\rightarrow$ scalar

Task 11 — Covariance-style matrix $X^T X$

$X = \text{torch.randn}(100, 8)$ — Compute the (8,8) matrix $X^T X$ using einsum. Verify with $X.T @ X$.

Write your einsum string: _____ Expected answer: 'ni,nj->ij' $\rightarrow$ (8,8)

Task 12 — Multi-head attention value aggregation

$\text{attn} = \text{torch.randn}(2, 4, 16, 16)$ (softmaxed, B,H,N,N) $V = \text{torch.randn}(2, 4, 16, 32)$ (B,H,N,D) — Aggregate values: for each query position i , compute a weighted sum of value vectors over all key positions j .

Write your einsum string: _____ Expected answer: 'bhij,bhjd->bhid' $\rightarrow$ (2,4,16,32)

4 Quick-Reference Cheat Sheet

Operation	NumPy/Manual	einsum string	Shape
Dot product (1D)	<code>np.dot(a, b)</code>	<code>'i,i->'</code>	$(n),(n) \rightarrow \text{scalar}$
Matrix-vector mult.	<code>A @ v</code>	<code>'ij,j->i'</code>	$(m,n),(n) \rightarrow (m,)$
Matrix-matrix mult.	<code>A @ B</code>	<code>'ij,jk->ik'</code>	$(m,n),(n,p) \rightarrow (m,p)$
Element-wise product	<code>a * b</code>	<code>'ij,ij->ij'</code>	$(m,n),(m,n) \rightarrow (m,n)$
Outer product	<code>np.outer(a,b)</code>	<code>'i,j->ij'</code>	$(m),(n) \rightarrow (m,n)$
Matrix trace	<code>np.trace(A)</code>	<code>'ii->'</code>	$(n,n) \rightarrow \text{scalar}$
Row-wise sum	<code>A.sum(axis=1)</code>	<code>'ij->i'</code>	$(m,n) \rightarrow (m,)$
Column-wise sum	<code>A.sum(axis=0)</code>	<code>'ij->j'</code>	$(m,n) \rightarrow (n,)$
Transpose	<code>A.T</code>	<code>'ij->ji'</code>	$(m,n) \rightarrow (n,m)$
Batch matmul	<code>torch.bmm(A,B)</code>	<code>'bij,bjk->bik'</code>	$(b,m,n),(b,n,p) \rightarrow (b,m,p)$
Inner product (2D)	<code>np.sum(A*B)</code>	<code>'ij,ij->'</code>	$(m,n),(m,n) \rightarrow \text{scalar}$
Diagonal extraction	<code>np.diag(A)</code>	<code>'ii->i'</code>	$(n,n) \rightarrow (n,)$
Hadamard + sum	<code>np.sum(a*b*c)</code>	<code>'i,i,i->'</code>	$(n),(n),(n) \rightarrow \text{scalar}$
Multi-head attention QK	<code>(Q @ K.T)</code>	<code>'bhid,bhjd->bhij'</code>	$(b,h,n,d) \times 2 \rightarrow (b,h,n,n)$
Tensor contraction	custom loop	<code>'ijk,jkl->il'</code>	$(a,b,c),(b,c,d) \rightarrow (a,d)$
Bilinear form	<code>x.T @ W @ y</code>	<code>'i,ij,j->'</code>	$(n),(n,m),(m,) \rightarrow \text{scalar}$
Covariance matrix	<code>(X-mu).T@(X-mu)/n</code>	<code>'ni,nj->ij'</code>	$(n,d),(n,d) \rightarrow (d,d)$
Scaled dot-product	<code>Q@K.T/sqrt(d)</code>	<code>'...id,...jd->...ij'</code>	$(...,n,d) \times 2 \rightarrow (...,n,n)$

Keep this page open while working on Chapter 5.

5 Advanced Patterns & Sharp Edges

5.1 The ellipsis operator '...'

Use `...` to handle arbitrary batch dimensions without naming them. Essential for writing shape-agnostic code.

```
# Works for any number of leading batch dims
Q = torch.randn(2, 4, 8, 16, 32) # (B, ..., N, D)
K = torch.randn(2, 4, 8, 16, 32)

scores = torch.einsum('...id,...jd->...ij', Q, K)
print(scores.shape) # torch.Size([2, 4, 8, 16, 16])
```

5.2 Three-operand einsum

NumPy and PyTorch both support multiple operands in a single call, but **they evaluate left-to-right pairwise**. For performance-critical code, use `opt_einsum` to find the optimal contraction order.

```
# Bilinear form in one call
x = torch.randn(4); W = torch.randn(4,4); y = torch.randn(4)
s = torch.einsum('i,ij,j->', x, W, y) # scalar

# opt_einsum for large contractions
# pip install opt-einsum
import opt_einsum as oe
result = oe.contract('bhid,bhjd->bhij', Q, K)
```

5.3 Performance: when NOT to use einsum

For the common cases — `matmul`, `bmm` — the vendor-optimised cuBLAS/cuDNN kernels launched by `@` or `torch.bmm` are typically 10–30% faster than `einsum` on GPU because they bypass the overhead of the `einsum` parser and contraction planner.

■ Profile before you optimize. Use `einsum` for clarity first; replace with `@` / `bmm` only where `timeit` proves a bottleneck.

5.4 Common mistakes

- **Wrong output order:** `'ij,jk->ki'` gives the *transpose* of what you expect from `matmul`. Always draw out axes before typing.
- **Implicit mode:** `np.einsum('ij,jk', A, B)` (no arrow) sorts free indices alphabetically and may surprise you. Always use explicit output.
- **Shape mismatch on contracted axis:** `'ij,kj->ik'` contracts over `j` — `A.shape[1]` must equal `B.shape[1]`. `Einsum` won't transpose `B` automatically.
- **Memory explosion:** naive three-operand contractions can create huge intermediates. If input tensors are large, use `opt_einsum` or manual two-step contractions.

6 Challenge Problems

These require combining einsum with other PyTorch ops. Write clean, runnable code. No partial credit for 'I know how it works'.

Challenge 1 — Gram Matrix Kernel

Given a feature matrix X of shape (N, D) , compute the $N \times N$ Gram matrix G where $G[i, j] = X[i] \cdot X[j]$. Then compute the squared Euclidean distance matrix $D[i, j] = \|x_i - x_j\|^2 = G[i, i] + G[j, j] - 2 \cdot G[i, j]$ entirely with einsum and broadcasting (no loops, no cdist). Verify with `torch.cdist`.

Your solution (code + einsum string + brief explanation):

Challenge 2 — Attention in 10 Lines

Implement scaled dot-product attention from scratch using only einsum, `torch.softmax`, and standard arithmetic. Signature: `attention(Q, K, V) → (B, H, N, D)`. Match the output of `F.scaled_dot_product_attention` exactly (on CPU, no dropout).

Your solution (code + einsum string + brief explanation):

Challenge 3 — Batched Bilinear Layer

A bilinear layer computes: $\text{out}[b, k] = \sum_{i, j} x[b, i] \cdot W[k, i, j] \cdot y[b, j]$ for a batch of (x, y) pairs. Implement this as a single einsum call for x of shape (B, D) , y of shape (B, D) , W of shape (K, D, D) . Verify against `torch.nn.Bilinear`.

Your solution (code + einsum string + brief explanation):

Challenge 4 — Low-Rank Approximation Check

Compute the nuclear norm (sum of singular values) of a matrix A using only `torch.linalg.svd` and einsum (no `.norm` calls). Then compute the Frobenius norm squared via einsum and verify it equals the sum of squared singular values (Parseval).

Your solution (code + einsum string + brief explanation):

7 Debugging Einsum: a Checklist

Step 1 — Label every axis before writing the string

Write down the name and size of every axis of every tensor before touching the keyboard. e.g. Q: (b=2, h=4, i=16, d=32).

Step 2 — Identify free vs contracted indices

Mark each index F (free → appears in output) or C (contracted → does NOT appear in output). Every index must be one of these two.

Step 3 — Check axis sizes match on contracted dims

Contracted indices must have the same size in all tensors where they appear. np.einsum will raise an exception if they don't.

Step 4 — Verify output shape before running

Write out the expected output shape from the free indices. After running, assert result.shape == expected_shape as the first line of every test.

Step 5 — Build up incrementally

If a 3-operand contraction is wrong, split into two 2-operand contractions and check intermediate shapes.

7.1 Diagnostic snippet

```
def debug_einsum(string, *tensors, **kwargs):
    """Drop-in for einsum with shape printing."""
    import re
    inputs_str, output_str = string.split('->')
    input_subs = inputs_str.split(',')
    print(f"einsum: '{string}'")
    for i, (subs, t) in enumerate(zip(input_subs, tensors)):
        print(f" operand {i}: subscripts='{subs}' shape={tuple(t.shape)}")
    result = torch.einsum(string, *tensors, **kwargs)
    print(f" output: subscripts='{output_str}' shape={tuple(result.shape)}")
    return result

# Usage:
# C = debug_einsum('bhid,bhjd->bhij', Q, K)
```

8 Final Boss: Transformer Block in Pure Einsum

Rewrite the following conventional PyTorch transformer attention block replacing every `@` / `bmm` / `sum` / `tensor dot` call with `einsum`. Do not change the mathematical semantics. Output must match the original within $1e-5$.

```
# GIVEN: conventional implementation (do NOT modify)
import torch, torch.nn.functional as F

def attention_conventional(Q, K, V):
    # Q,K,V: (B, H, N, D)
    B, H, N, D = Q.shape
    scores = torch.bmm(
        Q.view(B*H, N, D),
        K.view(B*H, N, D).transpose(-1, -2)
    ).view(B, H, N, N) / D**0.5
    attn = F.softmax(scores, dim=-1)
    out = torch.bmm(
        attn.view(B*H, N, N),
        V.view(B*H, N, D)
    ).view(B, H, N, D)
    return out

# YOUR TASK: implement using einsum only (no bmm, no @, no .view)
def attention_einsum(Q, K, V):
    D = Q.shape[-1]
    # ← your code here
    pass

# Verify
B, H, N, D = 2, 4, 16, 32
Q = torch.randn(B,H,N,D); K = torch.randn(B,H,N,D); V = torch.randn(B,H,N,D)
ref = attention_conventional(Q, K, V)
out = attention_einsum(Q, K, V)
assert torch.allclose(ref, out, atol=1e-5), f"Mismatch: max err = {
(ref-out).abs().max()}"
print("PASSED")
```

Hint: two `einsum` calls — one for scores, one for the weighted sum of values. The `softmax` stays as `F.softmax`.

Good luck. Don't look at the answer until it passes.